

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Título del Práctica:

RoboMaze

PONDERACIÓN: 10 pts

 **Tiempo estimado: 15 hrs/min**

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Habilidad(es) blandas a formar.....	3
2. Resultado del Aprendizaje.....	3
2.1. Objetivo SMART.....	3
3. Enunciado de la Práctica.....	4
3.1 Descripción del problema a resolver.....	4
3.2 Alcance de la práctica.....	5
Restricciones.....	5
Lenguajes y herramientas.....	6
Requisitos mínimos del sistema.....	6
4. Entregables.....	7
5. Material de apoyo.....	8
6. Recursos y herramientas a utilizar.....	9
Software / Hardware.....	9
Plataformas.....	9

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Perseverancia	El estudiante analiza diferentes rutas y estrategias de búsqueda para resolver un problema de navegación, evaluando alternativas, corrigiendo errores y mejorando continuamente su solución hasta encontrar el camino adecuado hacia el objetivo.

1.2. Competencia(s)

Diseñar e implementar soluciones basadas en algoritmos de búsqueda en espacios de estados, utilizando estructuras de datos, grafos y técnicas de exploración para resolver problemas de navegación y toma de decisiones en entornos virtuales.

1.3. Habilidad(es) blandas a formar

La práctica le permitirá desarrollar las siguientes habilidades:

- Pensamiento lógico y analítico.
- Resolución de problemas.
- Atención al detalle.
- Capacidad de investigación.
- Organización y estructuración de soluciones.
- Pensamiento crítico para la evaluación de alternativas.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Desarrollar un sistema capaz de encontrar rutas dentro de un laberinto virtual utilizando algoritmos clásicos de búsqueda en inteligencia artificial.	El sistema deberá permitir resolver al menos 5 laberintos diferentes utilizando BFS y DFS, mostrando la ruta encontrada, la cantidad de nodos explorados y el tiempo de ejecución.	Implementando algoritmos de búsqueda, estructuras de datos adecuadas, una API REST en Python y una interfaz	Para fortalecer las competencias en resolución de problemas, modelado de espacios de estados, algoritmos de búsqueda y desarrollo de sistemas	Al finalizar la práctica y entregar el proyecto en la fecha establecida.

		gráfica para la visualización del entorno y los resultados.	inteligentes.	
--	--	---	---------------	--

3. Enunciado de la Práctica

3.1 Descripción del problema a resolver

En múltiples aplicaciones de inteligencia artificial, robótica, videojuegos y sistemas autónomos, es necesario que un agente sea capaz de desplazarse dentro de un entorno desconocido o parcialmente conocido para alcanzar un objetivo específico. Este tipo de problemas requiere la exploración de diferentes alternativas y la selección de rutas que permitan llegar al destino de manera eficiente.

Para resolver este tipo de situaciones, se utilizan algoritmos de búsqueda en espacios de estados, los cuales permiten modelar un entorno como un conjunto de posiciones conectadas y determinar posibles caminos desde un punto inicial hasta una meta. Entre los algoritmos clásicos más utilizados se encuentran Breadth-First Search (BFS) y Depth-First Search (DFS), los cuales implementan diferentes estrategias de exploración para encontrar soluciones.

Ante esta necesidad, se propone el desarrollo de RoboMaze, un sistema capaz de representar laberintos virtuales y permitir que un agente inteligente encuentre una ruta desde una posición inicial hasta una posición objetivo utilizando algoritmos de búsqueda.

La solución deberá contar con una interfaz gráfica que permita visualizar el entorno, definir obstáculos, seleccionar puntos de inicio y destino, ejecutar distintos algoritmos de búsqueda y mostrar los resultados obtenidos. Además, el sistema deberá presentar información relevante sobre el proceso de búsqueda, incluyendo la ruta encontrada, la cantidad de nodos explorados y el tiempo de ejecución de cada algoritmo.

La práctica busca que el estudiante aplique conceptos fundamentales de inteligencia artificial relacionados con representación de estados, búsqueda en grafos, exploración de espacios de soluciones y análisis comparativo de algoritmos, construyendo una solución interactiva y funcional orientada a la resolución de problemas de navegación.

3.2 Alcance de la práctica

La práctica deberá incluir como mínimo los siguientes componentes:

Obligatorios

- Desarrollo de una aplicación capaz de representar un laberinto mediante una cuadrícula bidimensional.
- Implementación de un algoritmo Breadth-First Search (BFS) para encontrar una ruta entre un punto de inicio y un punto destino.
- Implementación de un algoritmo Depth-First Search (DFS) para encontrar una ruta entre un punto de inicio y un punto destino.
- Permitir la definición de una posición inicial y una posición objetivo dentro del laberinto.
- Permitir la colocación de obstáculos que bloqueen el paso del agente.
- Mostrar gráficamente el recorrido encontrado por cada algoritmo.
- Mostrar la ruta completa encontrada desde el origen hasta el destino.
- Mostrar la cantidad de nodos explorados por cada algoritmo.
- Mostrar el tiempo de ejecución de cada algoritmo.
- Permitir ejecutar BFS y DFS de forma independiente.
- Implementar un backend utilizando Python.
- Desarrollar una API REST para la ejecución de los algoritmos de búsqueda.
- Desarrollar una interfaz web que permita interactuar con el sistema.
- Implementar algún patrón de arquitectura para el backend.
- Uso de control de versiones mediante Git y repositorio en GitHub.
- Documentación básica de instalación y ejecución del sistema.

Opcional

- Generación automática de laberintos.
- Permitir modificar el tamaño del laberinto.
- Implementar el algoritmo A* para comparación de resultados.
- Mostrar animaciones del proceso de exploración de nodos.
- Permitir guardar y cargar laberintos previamente creados.
- Mostrar estadísticas comparativas entre BFS y DFS.
- Mostrar gráficas de rendimiento.
- Implementar múltiples puntos objetivo.
- Implementar obstáculos dinámicos durante la ejecución.
- Permitir exportar resultados a archivos CSV o PDF.

Restricciones

- Los algoritmos BFS y DFS deberán ser implementados por el estudiante; no se permite utilizar librerías que resuelvan automáticamente la búsqueda de rutas.
- El backend deberá desarrollarse exclusivamente utilizando Python.
- La lógica principal de búsqueda deberá ejecutarse en el backend.

- La interfaz gráfica únicamente deberá encargarse de la interacción y visualización de resultados.
- No se permite utilizar servicios externos de inteligencia artificial generativa para resolver el problema de búsqueda.
- No se permite utilizar bases de datos para almacenar el estado del laberinto o los resultados de búsqueda.

3.3 Requerimientos técnicos

Para el desarrollo de esta práctica se deberán utilizar las siguientes tecnologías:

Lenguajes y herramientas

- Python 3.x para el desarrollo del backend y la implementación de los algoritmos de búsqueda.
- Framework web de libre elección (FastAPI, Flask o equivalente).
- HTML, CSS y JavaScript para el desarrollo de la interfaz gráfica.
- Git para el control de versiones.
- GitHub para el almacenamiento y gestión del repositorio.
- Visual Studio Code o cualquier entorno de desarrollo equivalente.

Requisitos mínimos del sistema

- Implementación funcional del algoritmo Breadth-First Search (BFS).
- Implementación funcional del algoritmo Depth-First Search (DFS).
- Representación gráfica de un laberinto mediante una cuadrícula bidimensional.
- Definición de una posición inicial y una posición objetivo.
- Configuración de obstáculos dentro del laberinto.
- Visualización de la ruta encontrada por cada algoritmo.
- Visualización de la cantidad de nodos explorados durante la búsqueda.
- Visualización del tiempo de ejecución de cada algoritmo.
- API REST funcional para la ejecución de búsquedas.
- Interfaz web funcional para la interacción con el sistema.
- Comparación de resultados entre BFS y DFS.
- Manejo de errores cuando no exista una ruta válida entre el origen y el destino.
- Implementación de al menos un patrón de arquitectura para el backend.
- Evidencias de funcionamiento mediante capturas de pantalla.
- Documentación básica de instalación y ejecución.
- Repositorio GitHub con historial de cambios.
- El sistema deberá incluir al menos 5 laberintos predefinidos para realizar pruebas de funcionamiento.

4. Entregables

Tipo	Descripción
Repositorio del Proyecto	Enlace al repositorio GitHub que contenga el código fuente, documentación, historial de cambios y evidencias del desarrollo.
Backend	Proyecto desarrollado en Python encargado de la implementación de los algoritmos BFS y DFS, así como de la lógica de búsqueda y comunicación con el frontend.
API REST	Servicio REST funcional que permita recibir la configuración del laberinto y ejecutar los algoritmos de búsqueda solicitados por el usuario.
Frontend	Interfaz web que permita visualizar el laberinto, definir obstáculos, establecer el punto de inicio y destino, ejecutar búsquedas y mostrar resultados.
Implementación de BFS	Implementación completa del algoritmo Breadth-First Search para la resolución de rutas dentro del laberinto.
Implementación de DFS	Implementación completa del algoritmo Depth-First Search para la resolución de rutas dentro del laberinto.
Comparación de Algoritmos	Evidencia que muestre la comparación entre BFS y DFS utilizando métricas como longitud de ruta, nodos explorados y tiempo de ejecución.
Requerimientos Funcionales	Estas irán en el Manual Técnico. Describan las funcionalidades implementadas en el sistema.
Requerimientos No Funcionales	Estas irán en el Manual Técnico en formato Markdown (.md) y describirán aspectos relacionados con rendimiento, mantenibilidad, usabilidad y escalabilidad.
Patrón de Arquitectura	Deberá incluir un diagrama que represente el patrón de arquitectura utilizado en el backend y la interacción entre componentes.

Manual Técnico	Documento en formato Markdown (.md) que describa la arquitectura implementada, estructura del proyecto, algoritmos utilizados, API REST y posibles mejoras futuras.
Manual de Usuario	Documento en formato Markdown (.md) que explique la instalación, ejecución y uso del sistema mediante ejemplos e imágenes.
Evidencias de Funcionamiento	Capturas de pantalla que demuestren la creación de laberintos, ejecución de BFS, ejecución de DFS, visualización de rutas y comparación de resultados. Estos irán en el manual de usuario
Historial de Commits	El repositorio deberá contener un mínimo de 5 commits funcionales que evidencien el avance progresivo del desarrollo. No se aceptarán commits masivos realizados el mismo día que representen la totalidad del proyecto.
Repositorio del Proyecto	Enlace al repositorio GitHub que contenga el código fuente, documentación, historial de cambios y evidencias del desarrollo.

5. Material de apoyo

Se recomienda consultar los siguientes recursos para comprender los algoritmos de búsqueda, el desarrollo de APIs REST y la construcción de aplicaciones web para la visualización de resultados.

Documentación oficial

- Python: <https://docs.python.org/3>
- FastAPI: <https://fastapi.tiangolo.com>
- Flask: <https://flask.palletsprojects.com>
- Git: <https://git-scm.com/doc>
- GitHub: <https://docs.github.com>

Recursos sobre algoritmos de búsqueda

- Breadth-First Search (BFS):
https://en.wikipedia.org/wiki/Breadth-first_search
- Depth-First Search (DFS):
https://en.wikipedia.org/wiki/Depth-first_search
- Visualización interactiva de algoritmos:
<https://visualgo.net>
- Introducción a grafos y búsqueda:
<https://www.geeksforgeeks.org/graph-data-structure-and-algorithms>

6. Recursos y herramientas a utilizar

Software / Hardware

- Computadora personal con sistema operativo Windows, Linux o macOS.
- Python 3.11 o superior.
- Visual Studio Code o cualquier IDE equivalente.
- Git para control de versiones.
- Navegador web actualizado.
- Cliente para pruebas de APIs (Postman, Insomnia o equivalente).

Plataformas

- GitHub para alojamiento del repositorio y control de versiones.
- UEDI para la entrega de la práctica.

Se utilizará el mismo repositorio del curso

Fecha de entrega: 24/06/2026